Progressive Education Society's
MODERN COLLEGE OF ENGINEERING, Pune -05.
(An Autonomous Institute Affiliated to Savitribai Phule Pune University)
MCA Department

PRACTICAL SUBMISSION RECORD- A.Y. 2024-25

Class: SYMCA Div: B **Semester: III**	**Course Code: MCA1060** **Course Name: Laboratory Practice**		**Batch: S2**
Name: Ajit Bajarang Mali		**Roll No: 52140**	
CO No: NA		**Assignment No: Mini Project**	

Title: Smart Digit Teaching system

Code:

- requirements.txt

```
streamlit
streamlit-drawable-canvas
tensorflow
opencv-python
numpy
gTTS
```

- train_model.py

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.utils import to_categorical
import numpy as np

print("--- Starting Model Training with Dropout ---")

# 1. Load and Preprocess Data
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Reshape, Normalize, and One-Hot Encode
x_train = x_train.reshape(-1, 28, 28, 1).astype('float32') / 255
x_test = x_test.reshape(-1, 28, 28, 1).astype('float32') / 255
y_train = to_categorical(y_train, num_classes=10)
y_test = to_categorical(y_test, num_classes=10)

print(f"Training data shape: {x_train.shape}")

# 2. Define CNN Architecture
```

```
model = Sequential([
    Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=(28, 28, 1)),
    MaxPooling2D(pool_size=(2, 2)),

    Conv2D(64, kernel_size=(3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),

    Flatten(),

    # CRITICAL: Dropout layer added here to reduce confidence and improve grading variation
    Dropout(0.25),

    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

# 3. Compile and Train Model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# Training will take a few minutes
model.fit(x_train, y_train,
          epochs=10, # Increased epochs for better learning with Dropout
          batch_size=128,
          validation_data=(x_test, y_test),
          verbose=1)

# 4. Save Model
model.save('drawing_grader_cnn.h5')
print("\n--- Model trained successfully and saved as 'drawing_grader_cnn.h5' ---")
```

- app.py

```
# src/app.py (Updated with Canvas Clearing)

import streamlit as st
from streamlit_drawable_canvas import st_canvas
import tensorflow as tf
import numpy as np
import cv2
import random
import time
```

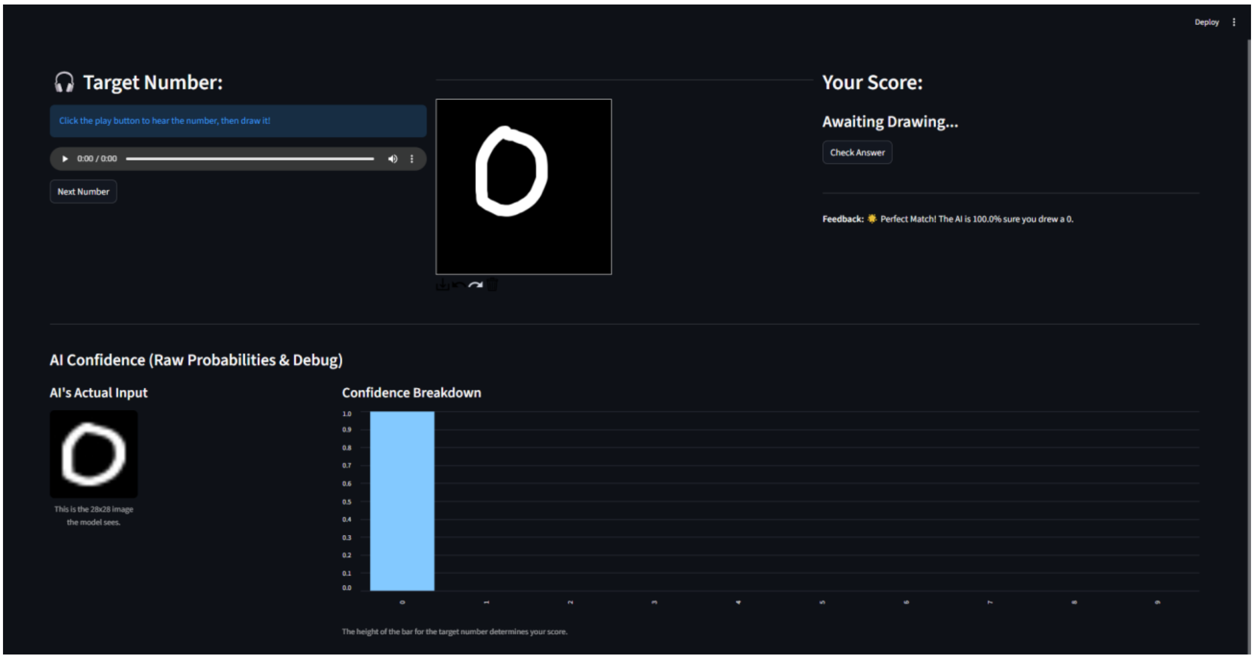
```
from gtts import gTTS
import io

# --- Configuration & Model Loading ---
st.set_page_config(layout="wide", page_title="AI Drawing Grader")

@st.cache_resource
def load_model():
    """Loads the trained CNN model once."""
    try:
        model = tf.keras.models.load_model('drawing_grader_cnn.h5')
        return model
    except Exception as e:
        st.error(f"Error loading model: {e}. Please run 'train_model.py' first.")
        return None

def generate_audio(number):
    """Generates speech audio for the given number and returns it as bytes."""
    tts = gTTS(str(number), lang='en', tld='com')
    audio_bytes = io.BytesIO()
    tts.write_to_fp(audio_bytes)
    audio_bytes.seek(0)
    return audio_bytes.read()

# --- MODIFIED: Initialize session state variables ---
if 'target_number' not in st.session_state:
    st.session_state.target_number = random.randint(0, 9)
    st.session_state.target_audio = generate_audio(st.session_state.target_number)

if 'score' not in st.session_state:
    st.session_state.score = "Awaiting Drawing..."
if 'debug_image_data' not in st.session_state:
    st.session_state.debug_image_data = None
if 'predictions' not in st.session_state:
    st.session_state.predictions = None
if 'feedback_message' not in st.session_state:
    st.session_state.feedback_message = ""
# --- NEW: Add a counter to reset the canvas key ---
if 'canvas_counter' not in st.session_state:
    st.session_state.canvas_counter = 0

model = load_model()
```

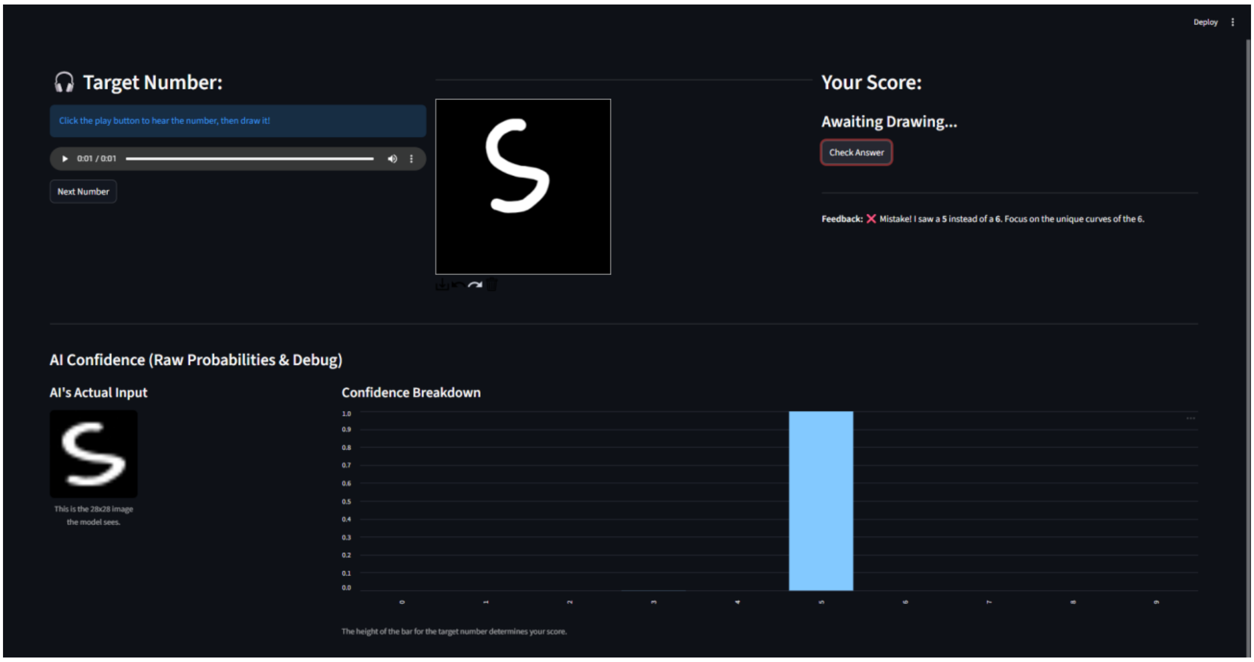
```
# --- Preprocessing Function (The Pipeline) - NO CHANGES HERE ---
def preprocess_image(canvas_data):
    if canvas_data is None:
        return None
    img = canvas_data[:, :, :3].astype('uint8')
    img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    _, thresh = cv2.threshold(img_gray, 20, 255, cv2.THRESH_BINARY)
    contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    if not contours:
        return None
    c = max(contours, key=cv2.contourArea)
    x, y, w, h = cv2.boundingRect(c)
    cropped_image = thresh[y:y+h, x:x+w]
    resized_image = cv2.resize(cropped_image, (20, 20), interpolation=cv2.INTER_AREA)
    final_image = np.zeros((28, 28), dtype=np.uint8)
    final_image[4:24, 4:24] = resized_image
    processed_for_model = final_image.astype('float32') / 255.0
    processed_for_model = processed_for_model.reshape(1, 28, 28, 1)
    st.session_state.debug_image_data = final_image.astype(np.uint8)
    return processed_for_model

# --- Scoring Logic - NO CHANGES HERE ---
def get_grade_and_feedback(predictions, target_number):
    predicted_digit = np.argmax(predictions[0])
    confidence = predictions[0][target_number]
    feedback_message = ""
    if predicted_digit == target_number:
        final_score = confidence * 100
        if confidence >= 0.95:
            feedback_message = f"□ Perfect Match! The AI is {confidence*100:.1f}% sure you drew a
{target_number}."
        elif confidence >= 0.80:
            feedback_message = f"□ Excellent! Great form, but you can tighten up your lines for a higher score."
        else:
            feedback_message = f"□ Correct digit! The AI only gave a {confidence*100:.1f}% confidence score.
Focus on filling the space!"
    else:
        final_score = confidence * 50
        feedback_message = f"□ Mistake! I saw a **{predicted_digit}** instead of a **{target_number}**.
Focus on the unique curves of the {target_number}."
    return round(final_score, 1), feedback_message

# --- Main Logic and Layout ---
```

```
col1, col2, col3 = st.columns([1, 1, 1])

with col1:
    st.markdown("## □ Target Number:")
    st.info("Click the play button to hear the number, then draw it!")
    st.audio(st.session_state.target_audio, format='audio/mp3')

    # --- MODIFIED: The button logic now also increments the canvas counter ---
    if st.button("Next Number", key='next_btn'):
        st.session_state.target_number = random.randint(0, 9)
        st.session_state.target_audio = generate_audio(st.session_state.target_number)
        st.session_state.score = "Awaiting Drawing..."
        st.session_state.debug_image_data = None
        st.session_state.predictions = None
        st.session_state.feedback_message = ""

        # --- NEW: Increment the counter to change the canvas key ---
        st.session_state.canvas_counter += 1

        st.rerun()

with col2:
    st.markdown("---")
    # --- MODIFIED: The key is now dynamic, tied to our counter ---
    canvas_result = st_canvas(
        fill_color="rgb(0, 0, 0)",
        stroke_width=20,
        stroke_color="rgb(255, 255, 255)",
        background_color="rgb(0, 0, 0)",
        height=300,
        width=300,
        key=f"canvas_{st.session_state.canvas_counter}",
    )

with col3:
    st.markdown("## Your Score:")
    if isinstance(st.session_state.score, float):
        st.markdown(f"### **{st.session_state.score}/100**")
    else:
        st.markdown(f"### **{st.session_state.score}**")

    if st.button("Check Answer", key='check_btn'):
```

```
        if model is None:
            st.warning("Model not loaded. Please check the console.")
        elif canvas_result.image_data is not None:
            processed_img = preprocess_image(canvas_result.image_data)
            if processed_img is not None:
                predictions = model.predict(processed_img)
                st.session_state.predictions = predictions[0]
                final_score, feedback = get_grade_and_feedback(
                    predictions, st.session_state.target_number)
                st.session_state.score = final_score
                st.session_state.feedback_message = feedback
            else:
                st.session_state.score = 0.0
                st.session_state.feedback_message = "□ Please draw a number clearly before checking!"

    st.markdown("---")
    st.markdown(f"**Feedback:** {st.session_state.feedback_message}")

# --- Bottom Debug Section (No changes here) ---
st.markdown("---")
st.markdown("### AI Confidence (Raw Probabilities & Debug)")

col4, col5 = st.columns([1, 3])

with col4:
    st.markdown("#### AI's Actual Input")
    if st.session_state.debug_image_data is not None:
        st.image(st.session_state.debug_image_data,
                 caption="This is the 28x28 image the model sees.",
                 width=150)

with col5:
    st.markdown("#### Confidence Breakdown")
    if st.session_state.predictions is not None:
        st.bar_chart(st.session_state.predictions, use_container_width=True)
        st.caption("The height of the bar for the target number determines your score.")
```

Output:


Deploy
Target Number:
Click the play button to hear the number, then draw it!
0:00 / 0:00
Next Number
Your Score:
Awaiting Drawing...
Check Answer
Feedback: Perfect Match! The AI is 100.0% sure you drew a 0.
AI Confidence (Raw Probabilities & Debug)
AI's Actual Input
This is the 28x28 image the model sees.
Confidence Breakdown
The height of the bar for the target number determines your score.


Deploy
Target Number:
Click the play button to hear the number, then draw it!
0:01 / 0:01
Next Number
Your Score:
Awaiting Drawing...
Check Answer
Feedback: ❌ Mistake! I saw a 5 instead of a 6. Focus on the unique curves of the 6.
AI Confidence (Raw Probabilities & Debug)
AI's Actual Input
This is the 28x28 image the model sees.
Confidence Breakdown
The height of the bar for the target number determines your score.